Data Structures and Algorithms

Lecture 10

Sorting Techniques

Part II

Dept. of ICT, Faculty of Technology- UoR, 2026

Merge Sort

Merge Sort

- Much more efficient than Bubble Sort and Selection Sort
- Involves Recursion
- Requires an additional array in memory
- The algorithm divides the array into two halves and applies a merge sort on each half recursively.
- After the two halves are sorted, merge them.

Divide and Conquer

The divide-and-conquer approach consists of three steps.

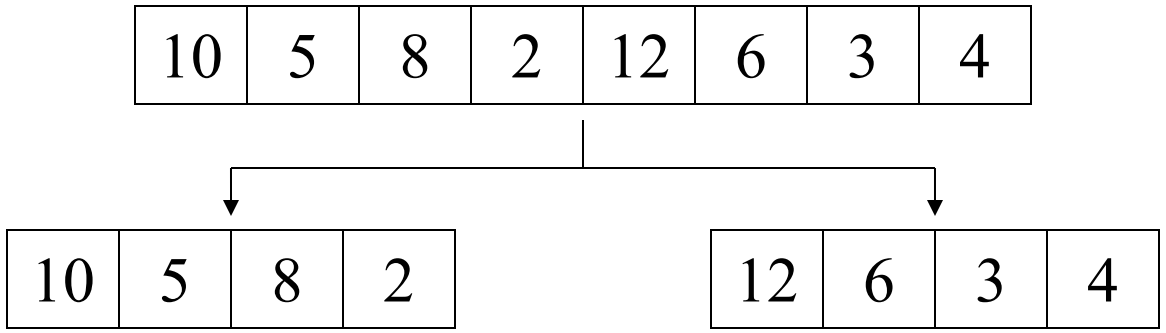
1. **Divide**: If the input size is smaller than a certain threshold (say, one or two elements), solve the problem directly using a straightforward method and return the solution. Otherwise, divide the input data into two or more disjoint subsets.
2. **Recur**: Recursively solve the subproblems associated with the subsets.
3. **Conquer**: Take the solutions to the subproblems and "merge" them into a solution to the original problem.

Merge Sort

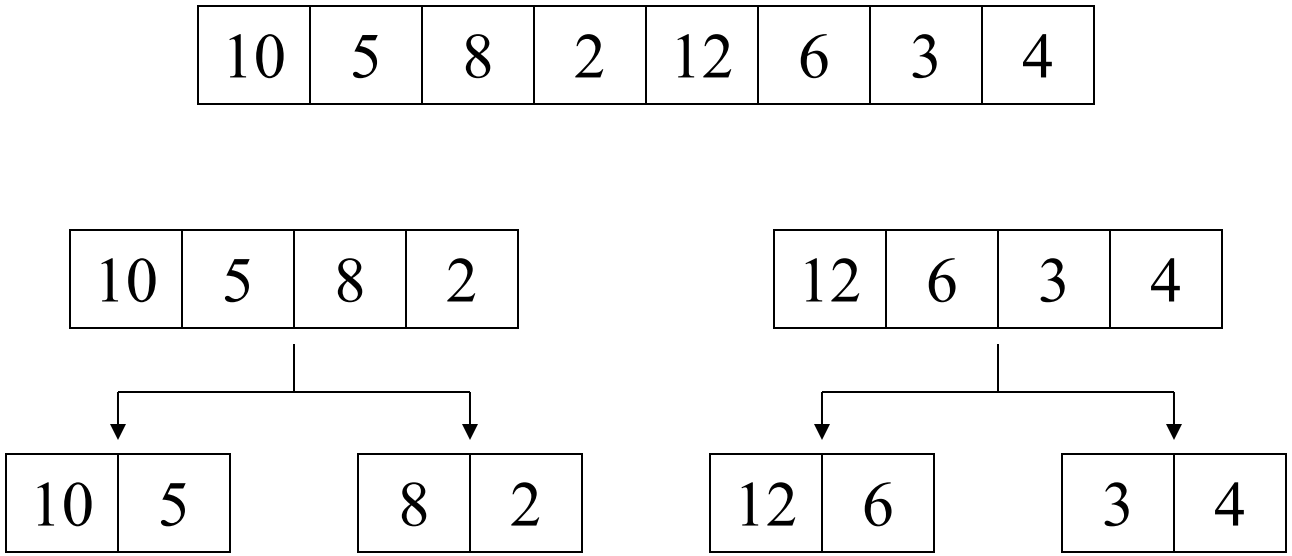
Example

10	5	8	2	12	6	3	4

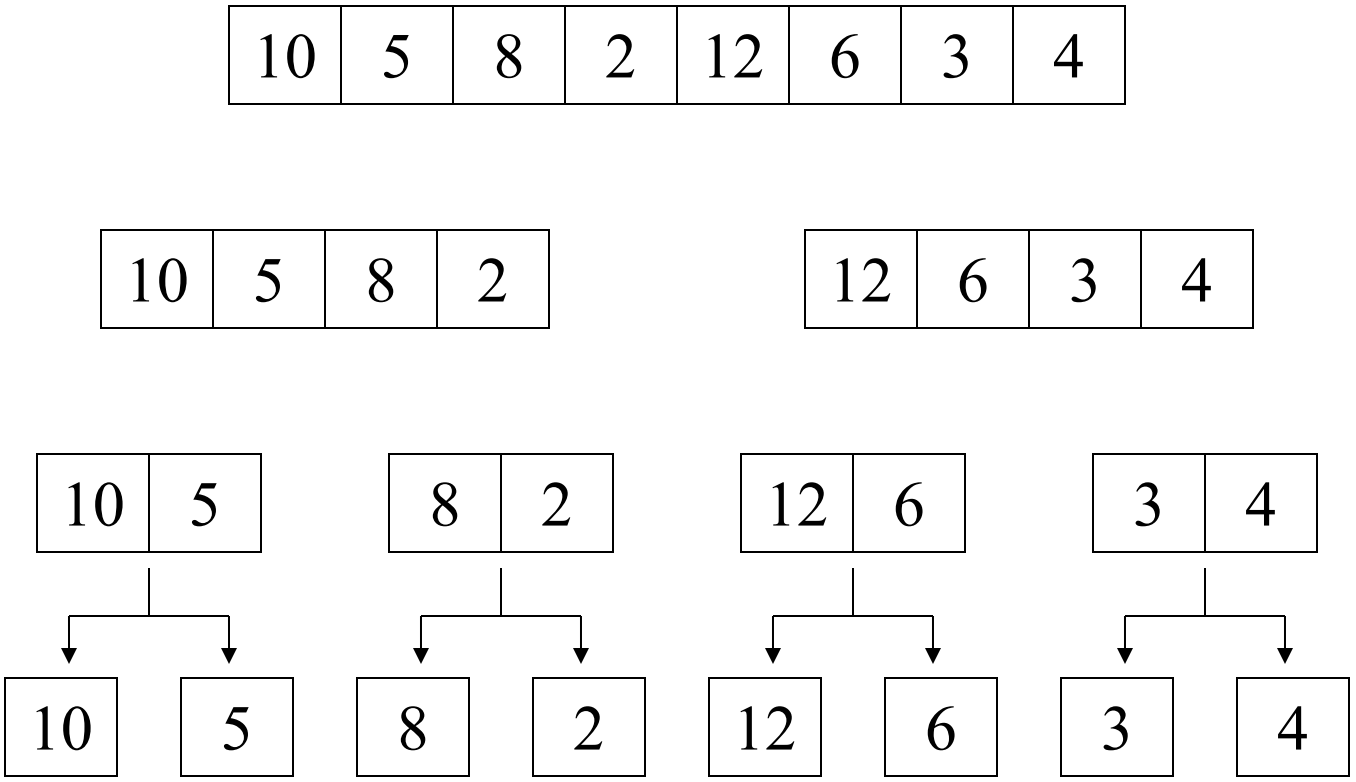
Merge Sort - Demo



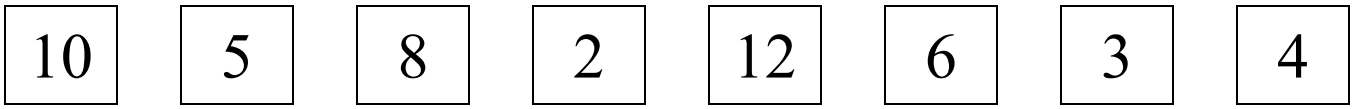
Merge Sort - Demo



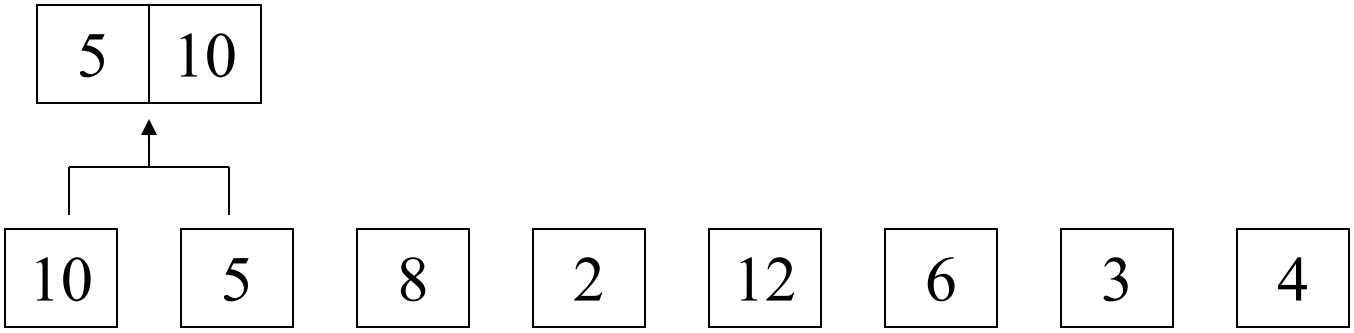
Merge Sort - Demo



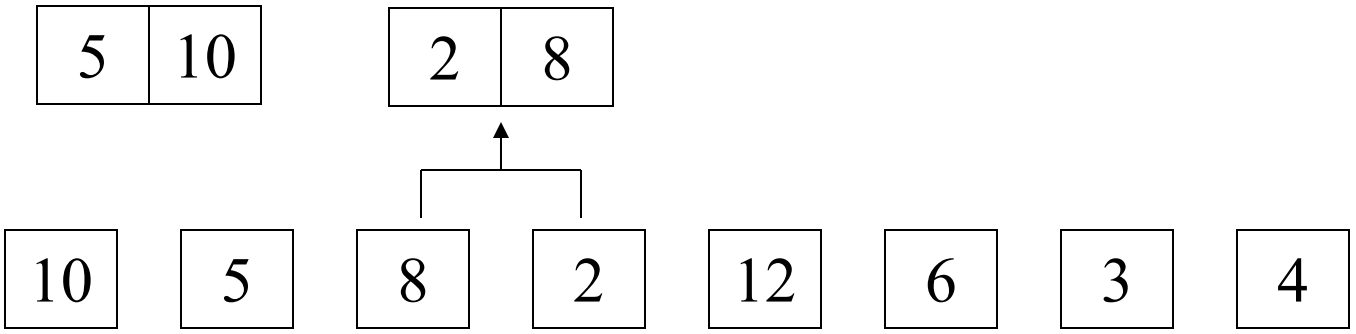
Merge Sort - Demo



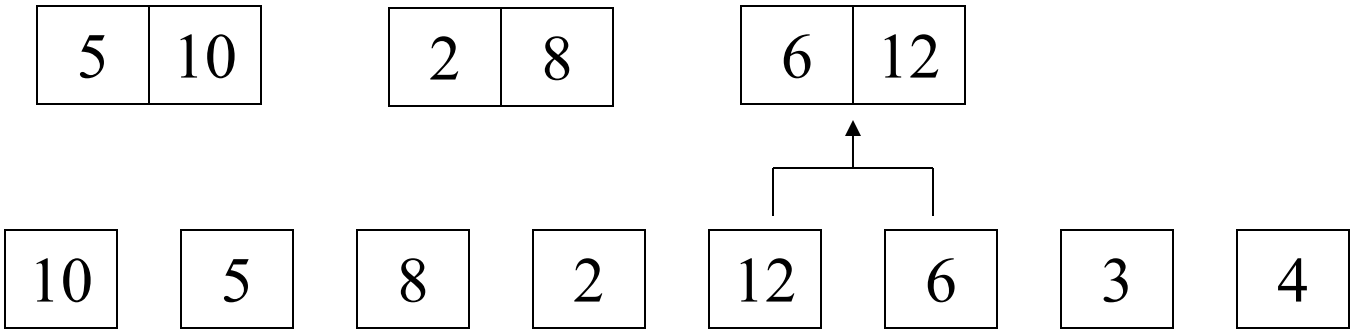
Merge Sort - Demo



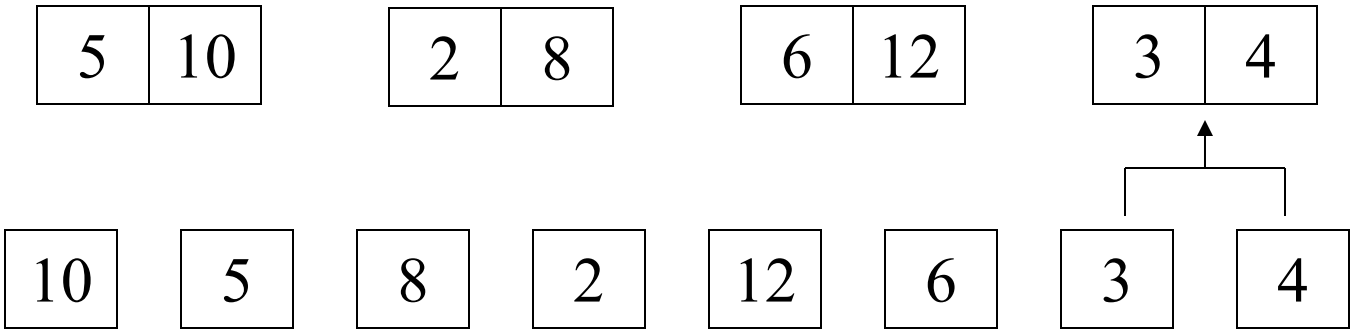
Merge Sort - Demo



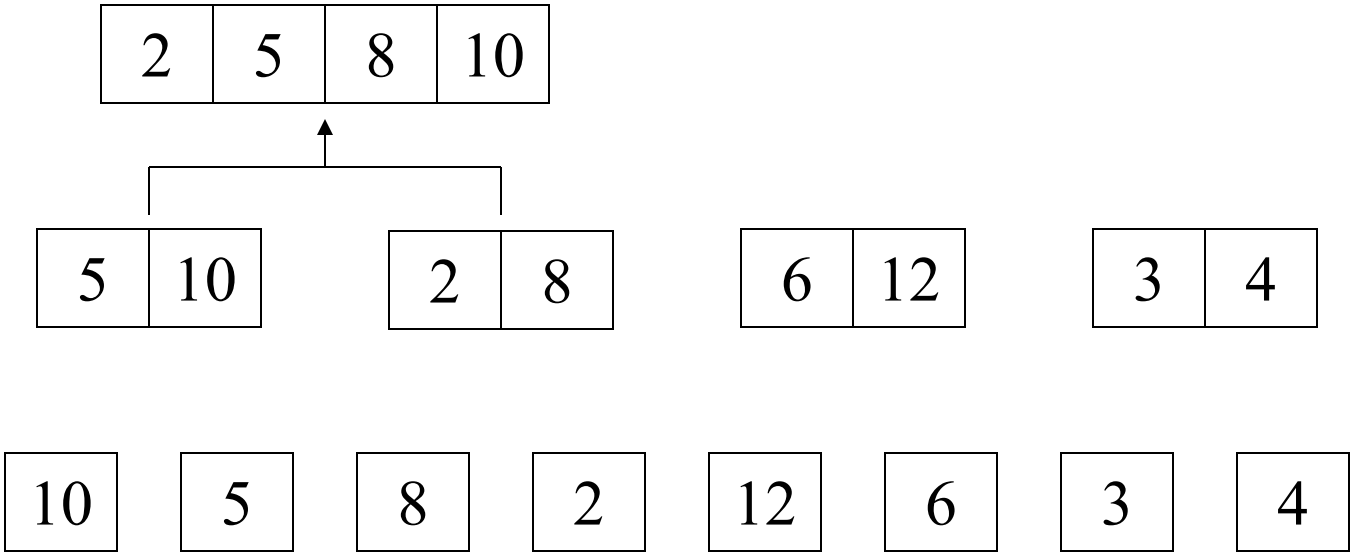
Merge Sort - Demo



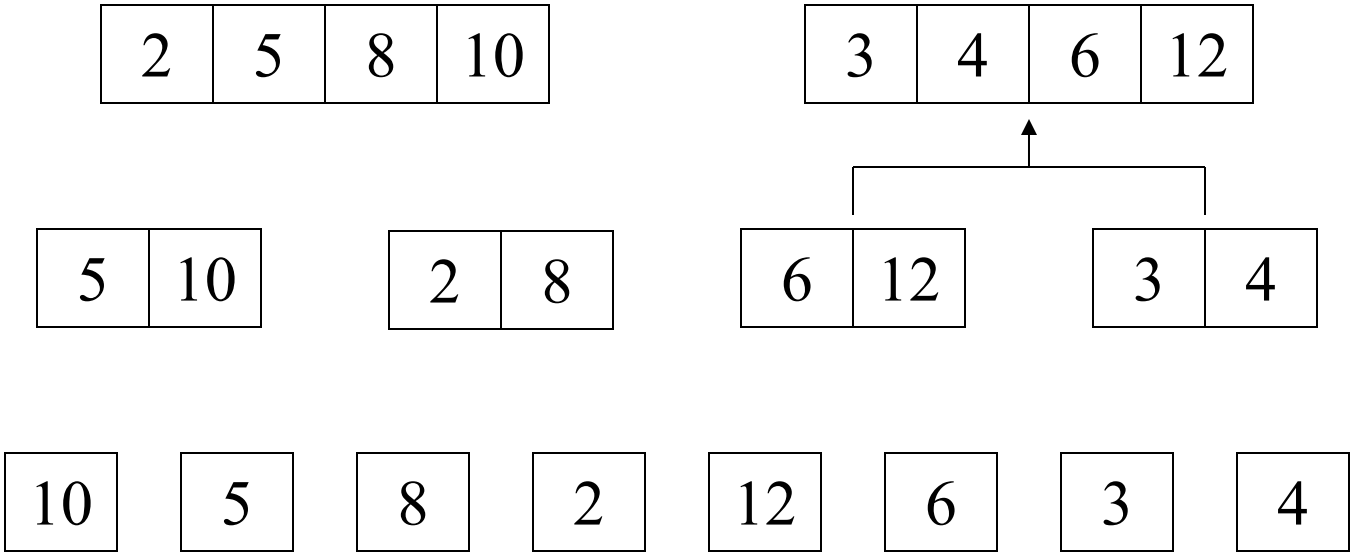
Merge Sort - Demo



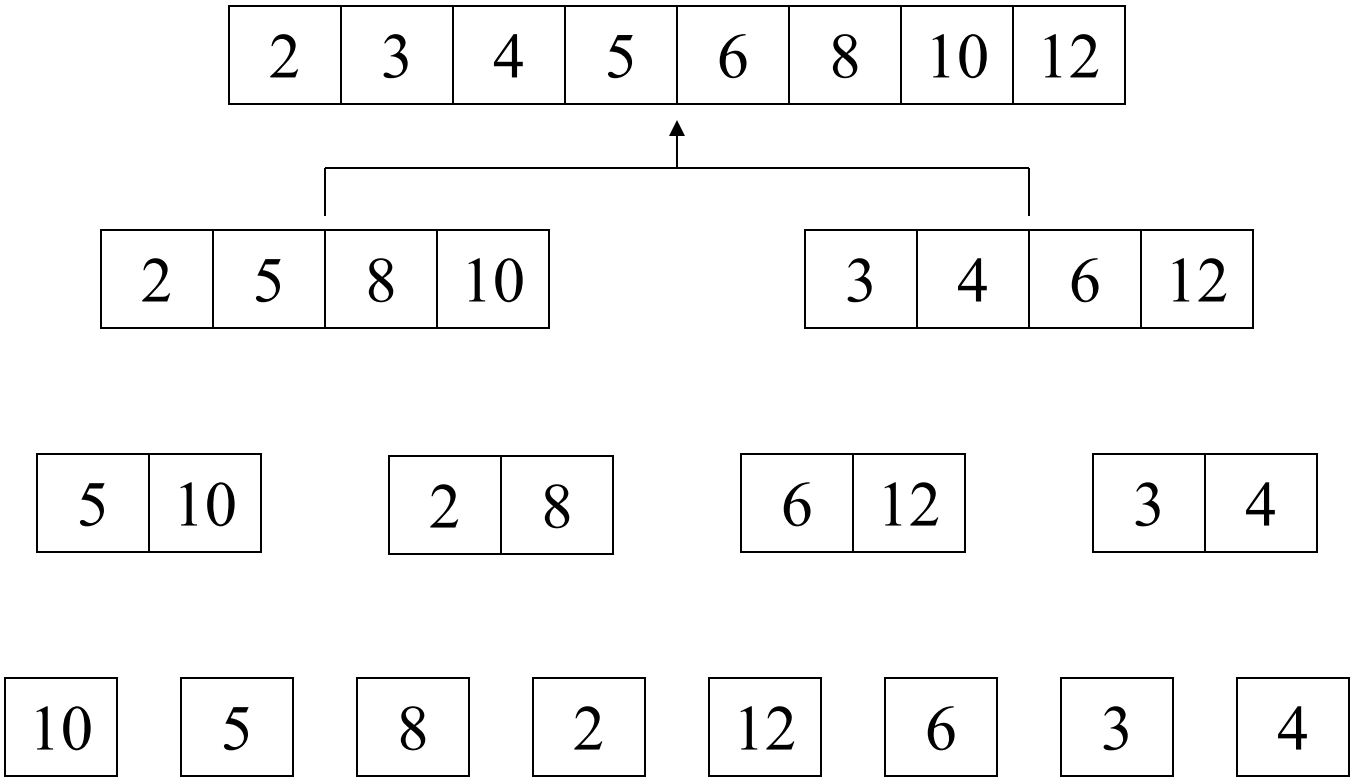
Merge Sort - Demo



Merge Sort - Demo



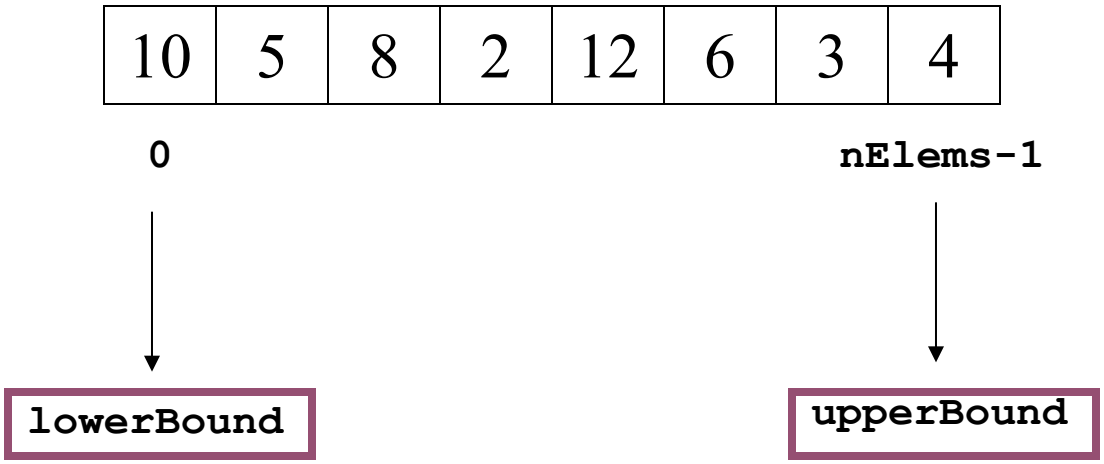
Merge Sort - Demo



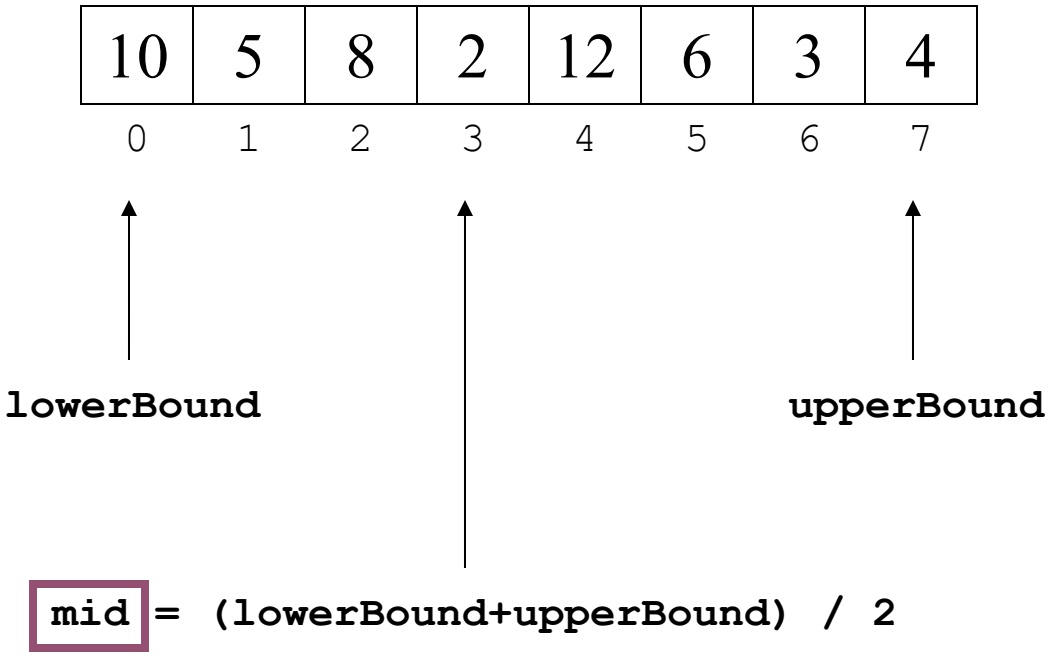
Merge Sort - Demo

2	3	4	5	6	8	10	12

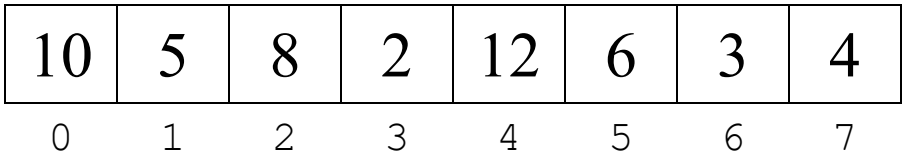
Merge Sort

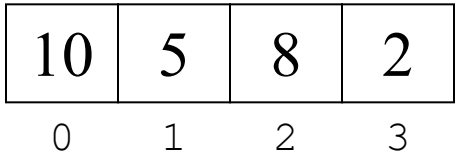


Merge Sort

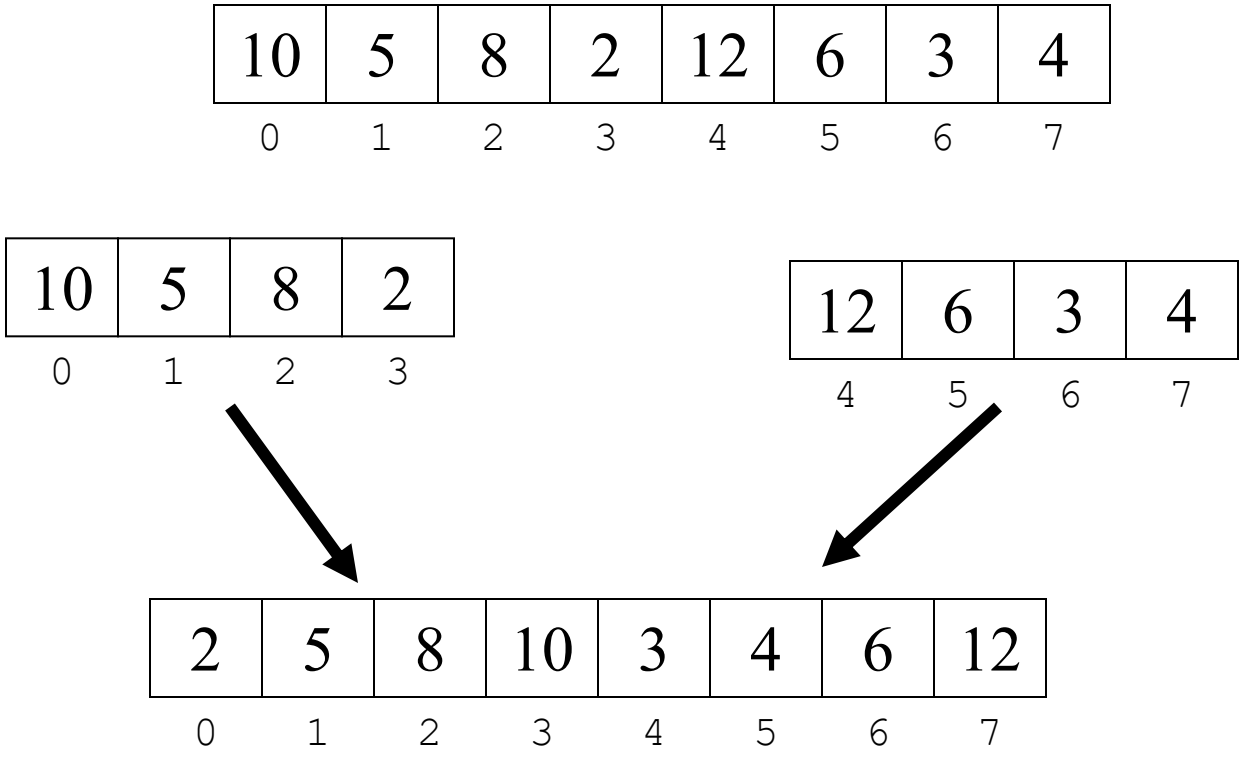

10 5 8 2 12 6 3 4
0 1 2 3 4 5 6 7
lowerBound
upperBound
mid = (lowerBound+upperBound) / 2

Merge Sort

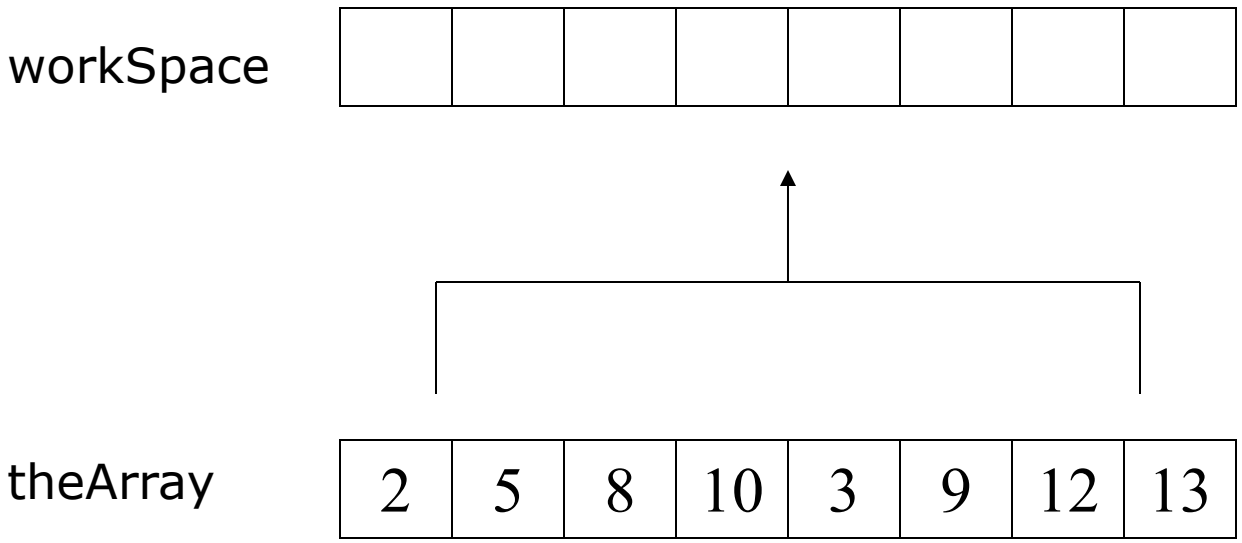




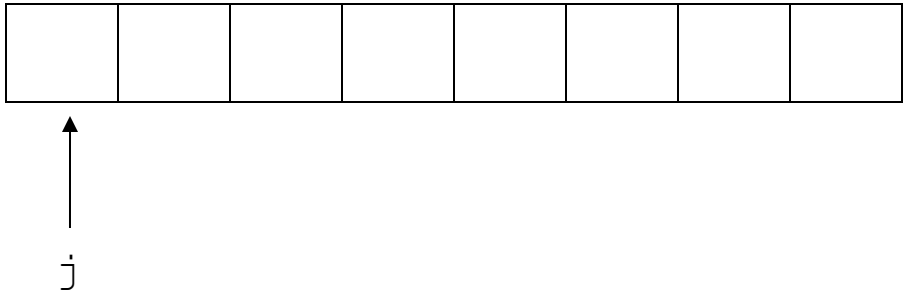
Merge Sort

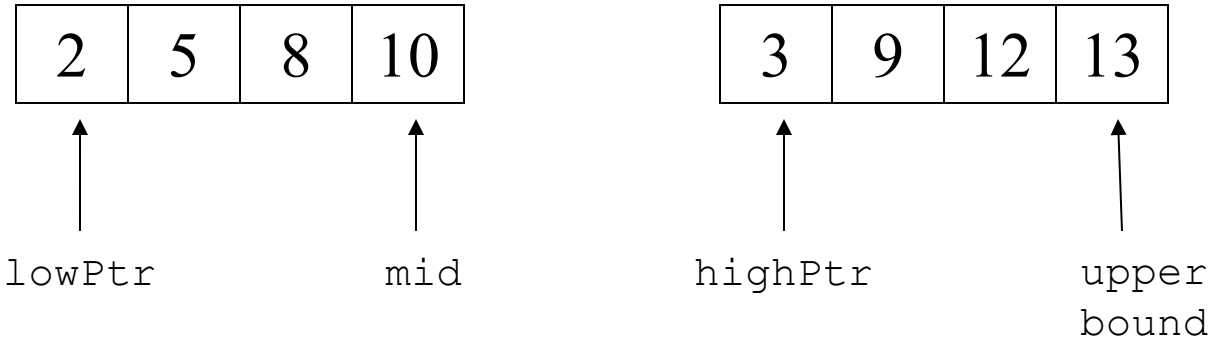

10 5 8 2 12 6 3 4
0 1 2 3 4 5 6 7
10 5 8 2
0 1 2 3
12 6 3 4
4 5 6 7
2 5 8 10 3 4 6 12
0 1 2 3 4 5 6 7

Merge Sort

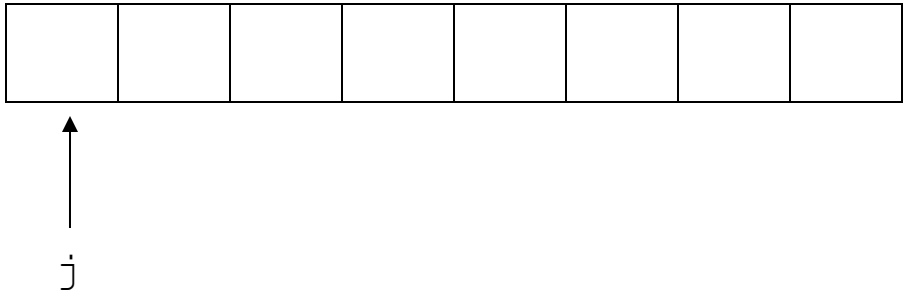


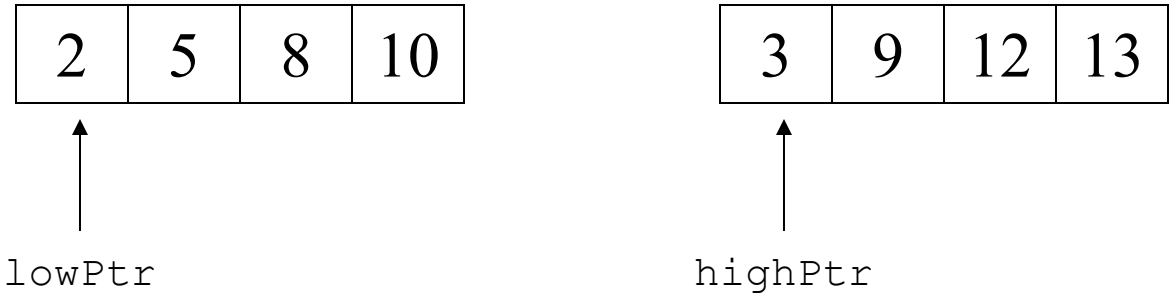
Merge Sort



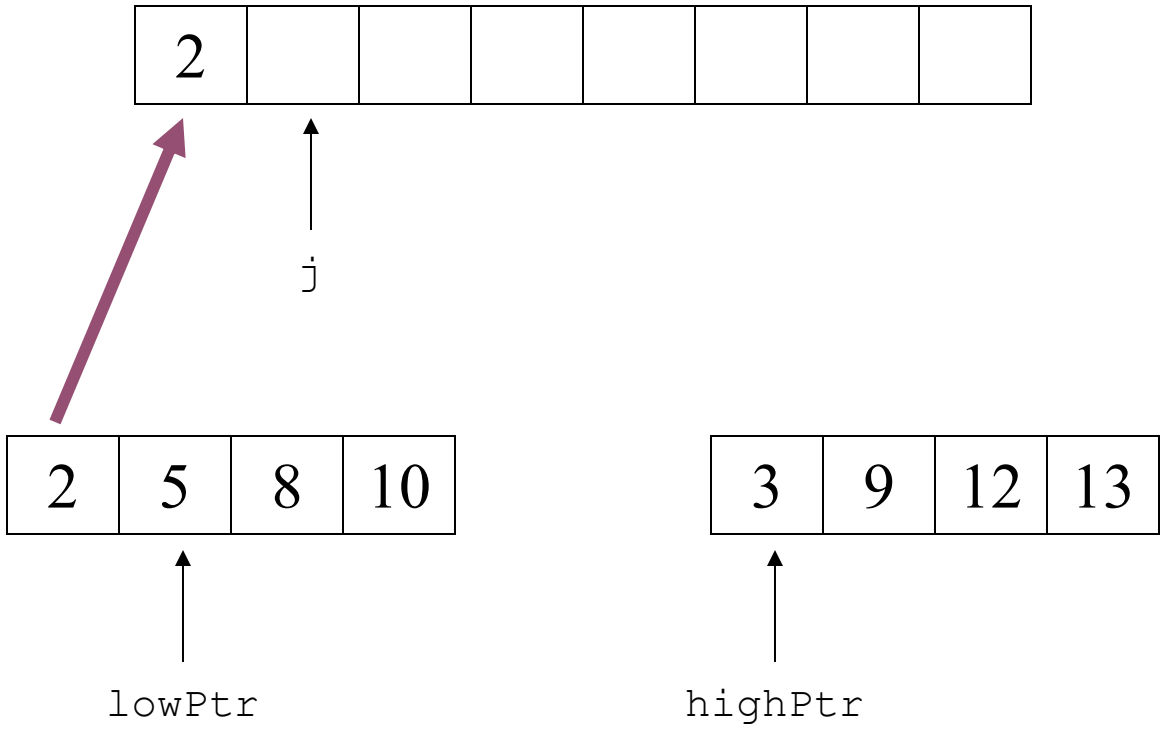


Merge Sort

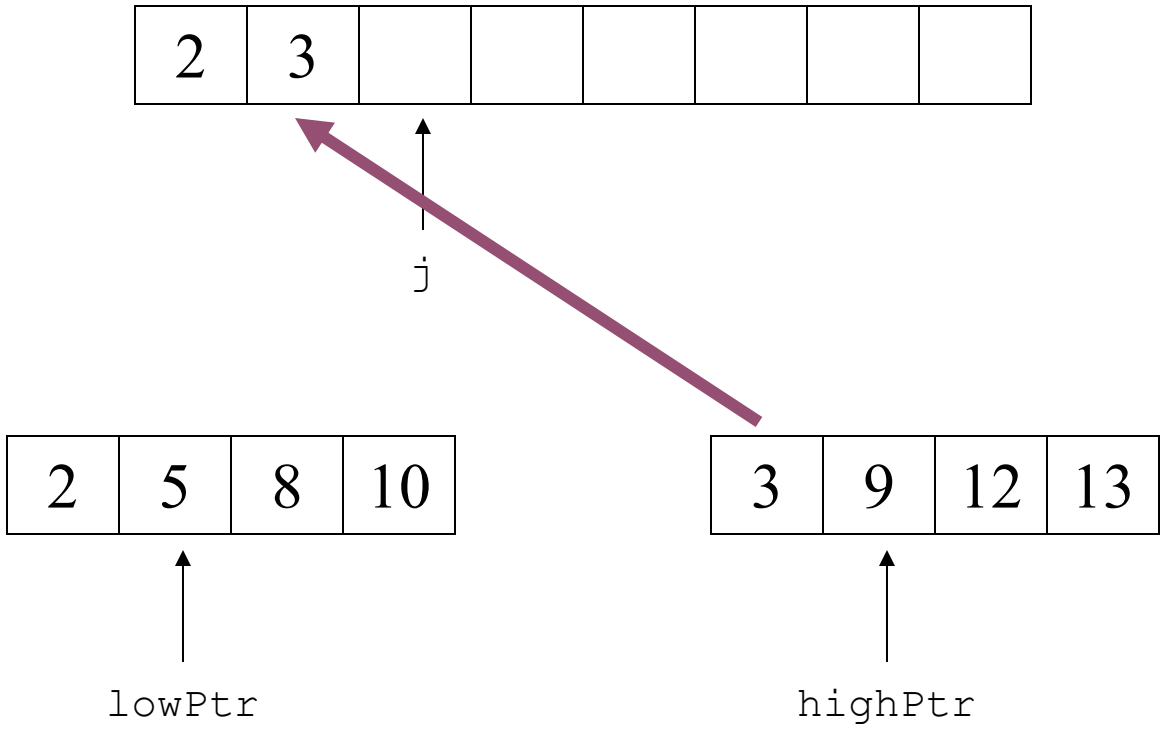




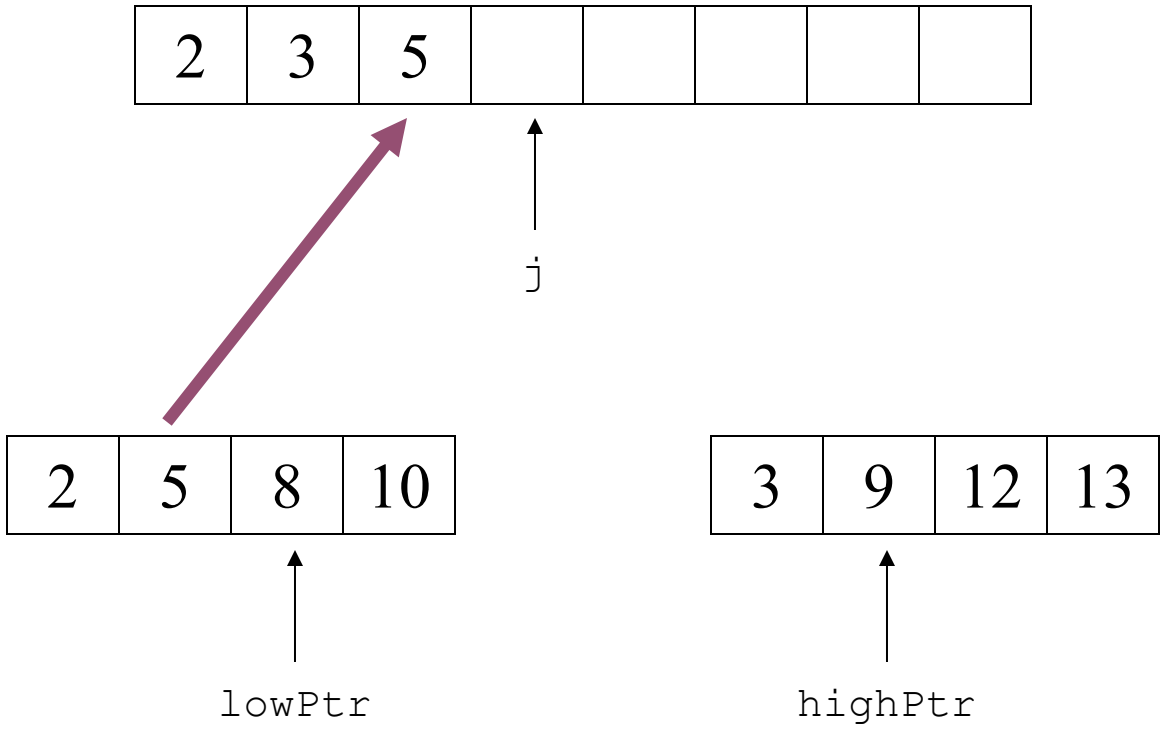
Merge Sort



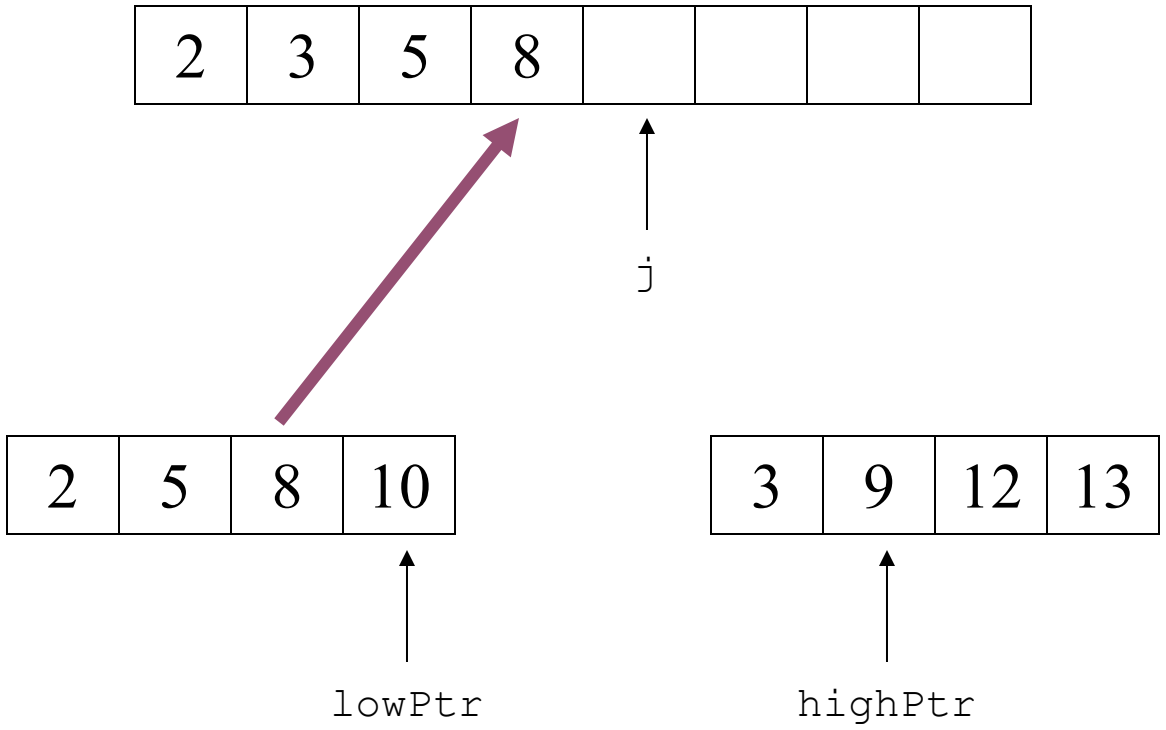
Merge Sort



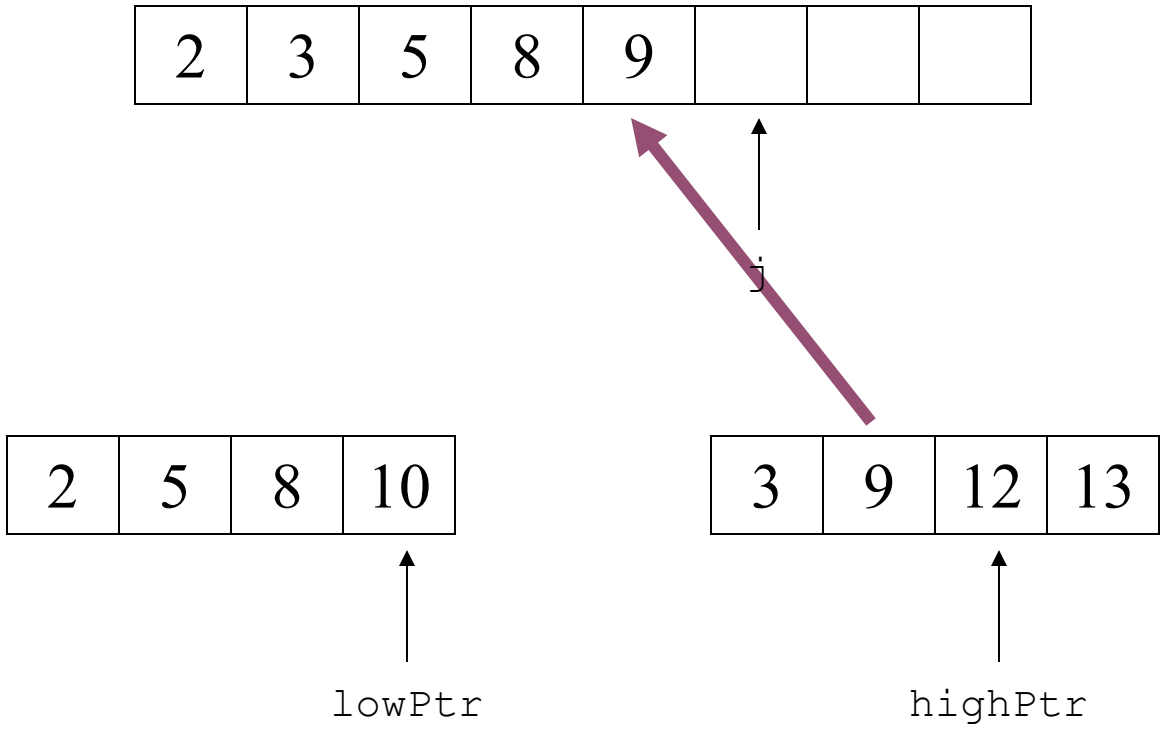
Merge Sort



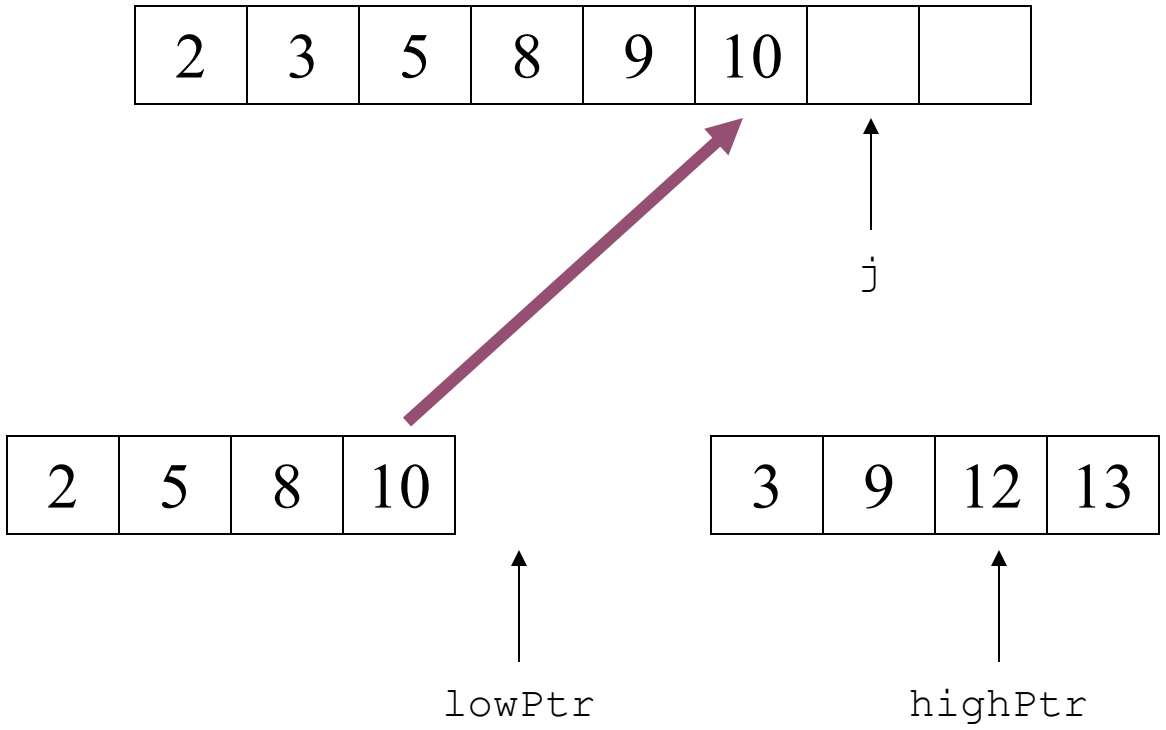
Merge Sort



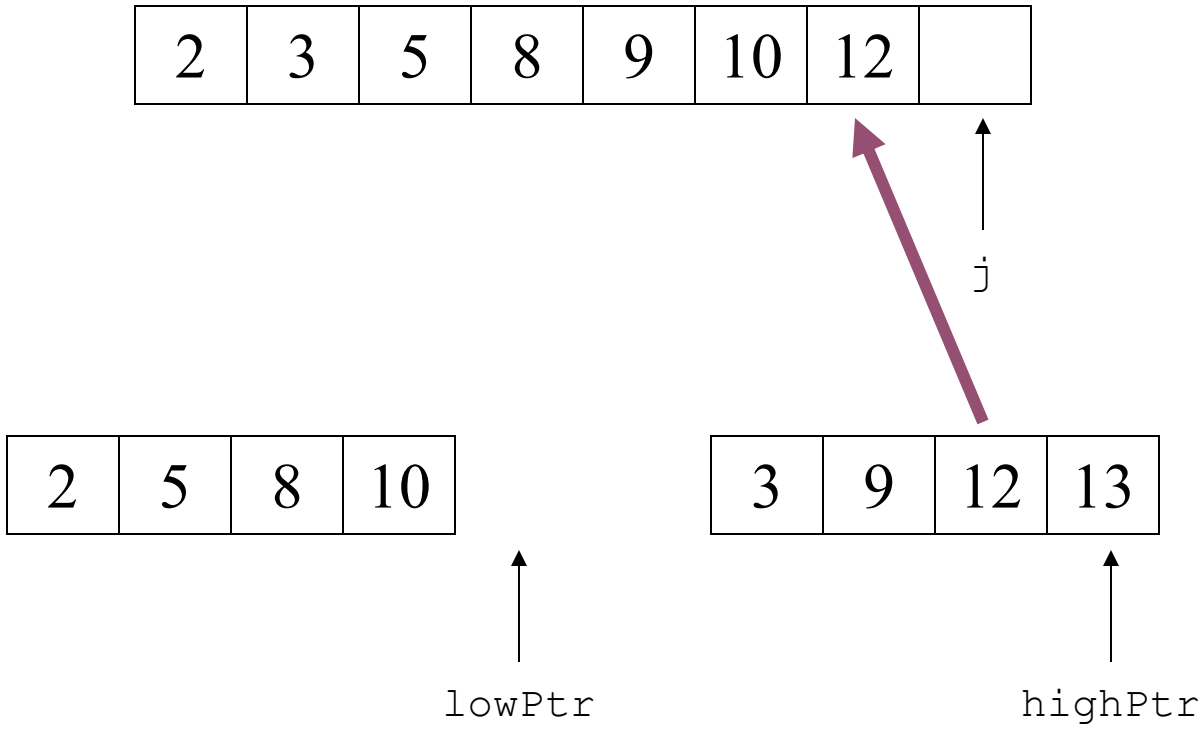
Merge Sort



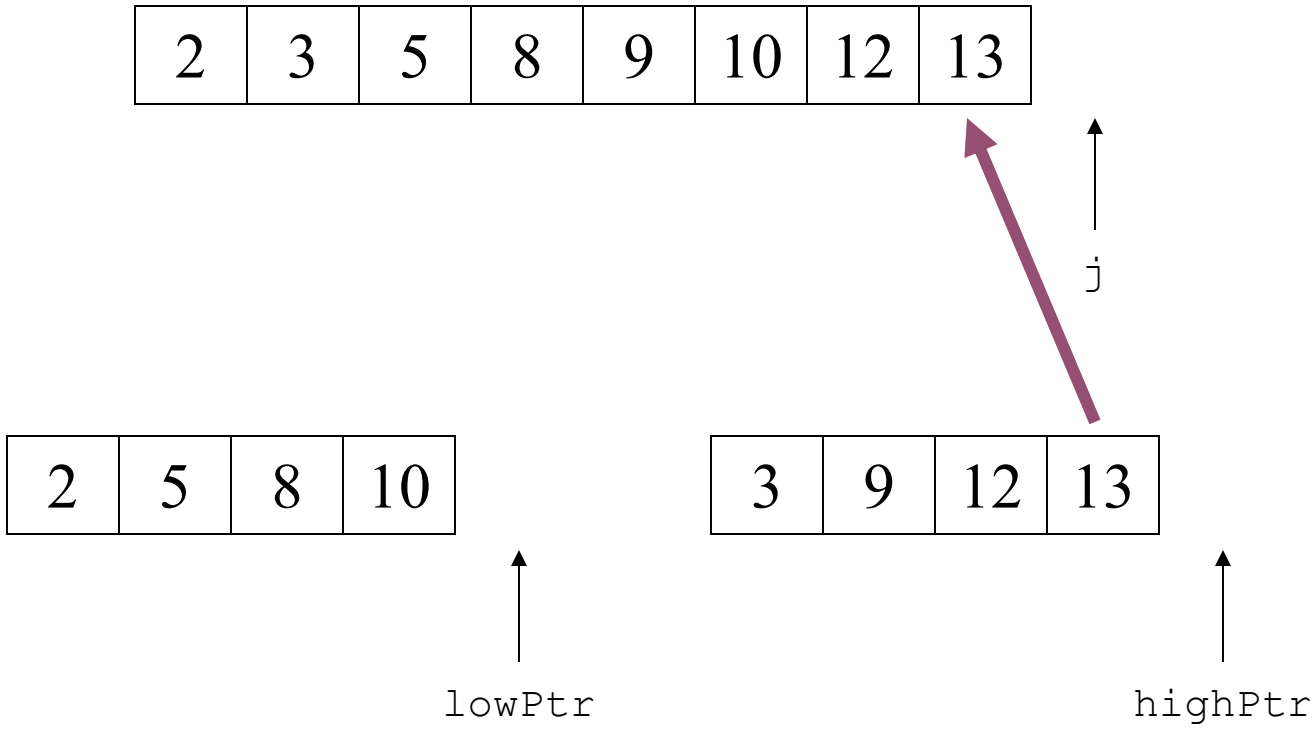
Merge Sort



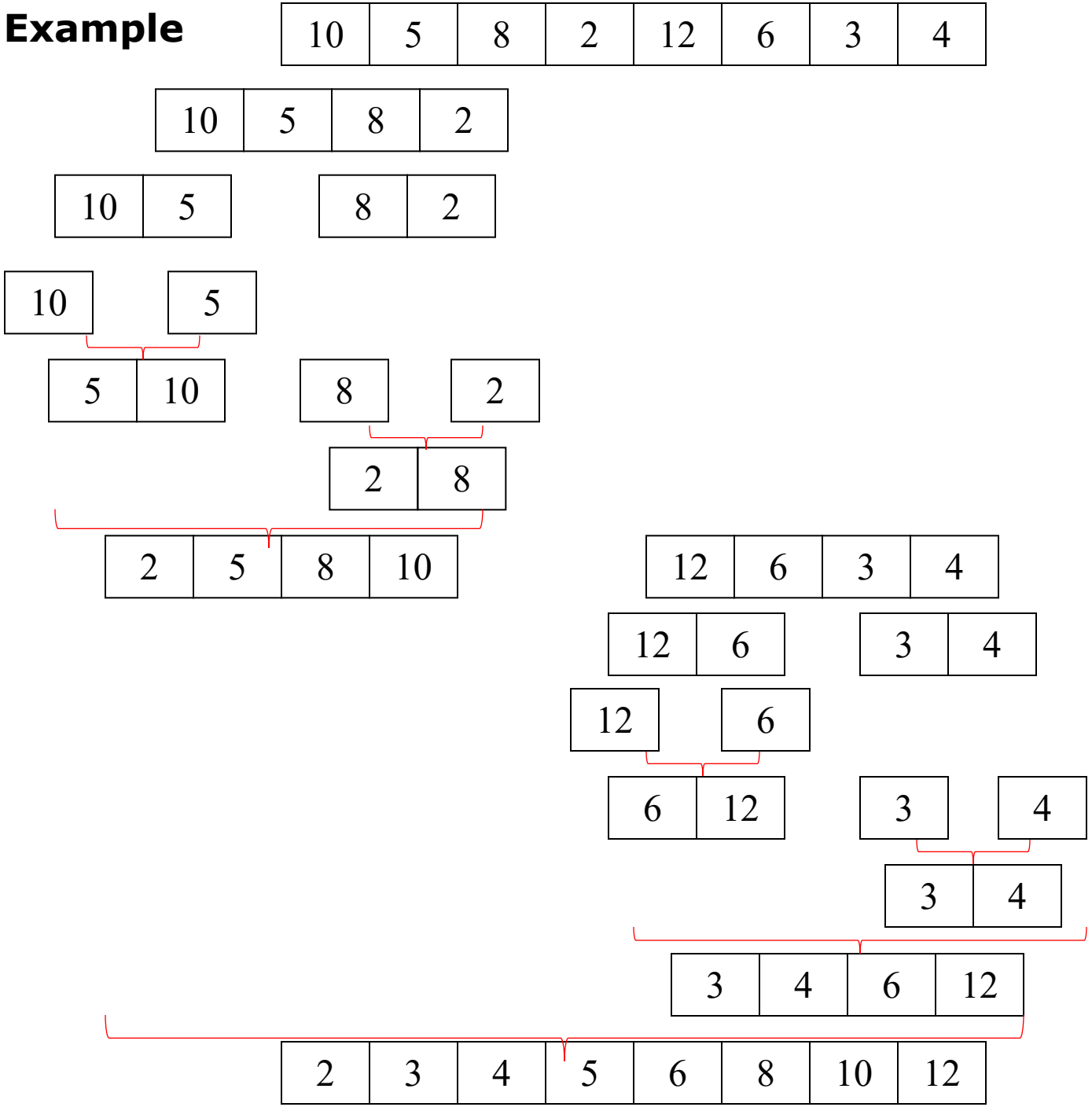
Merge Sort

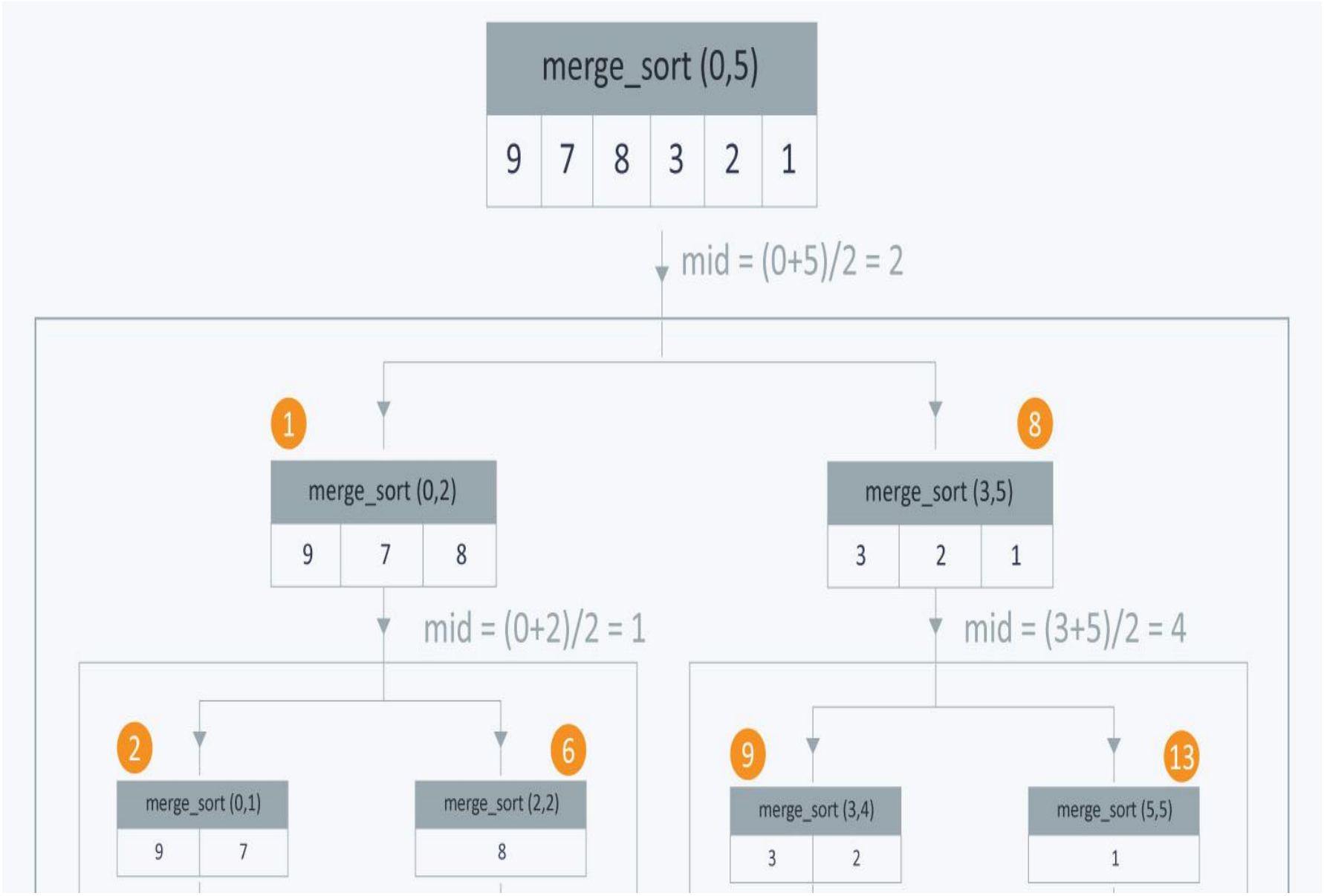


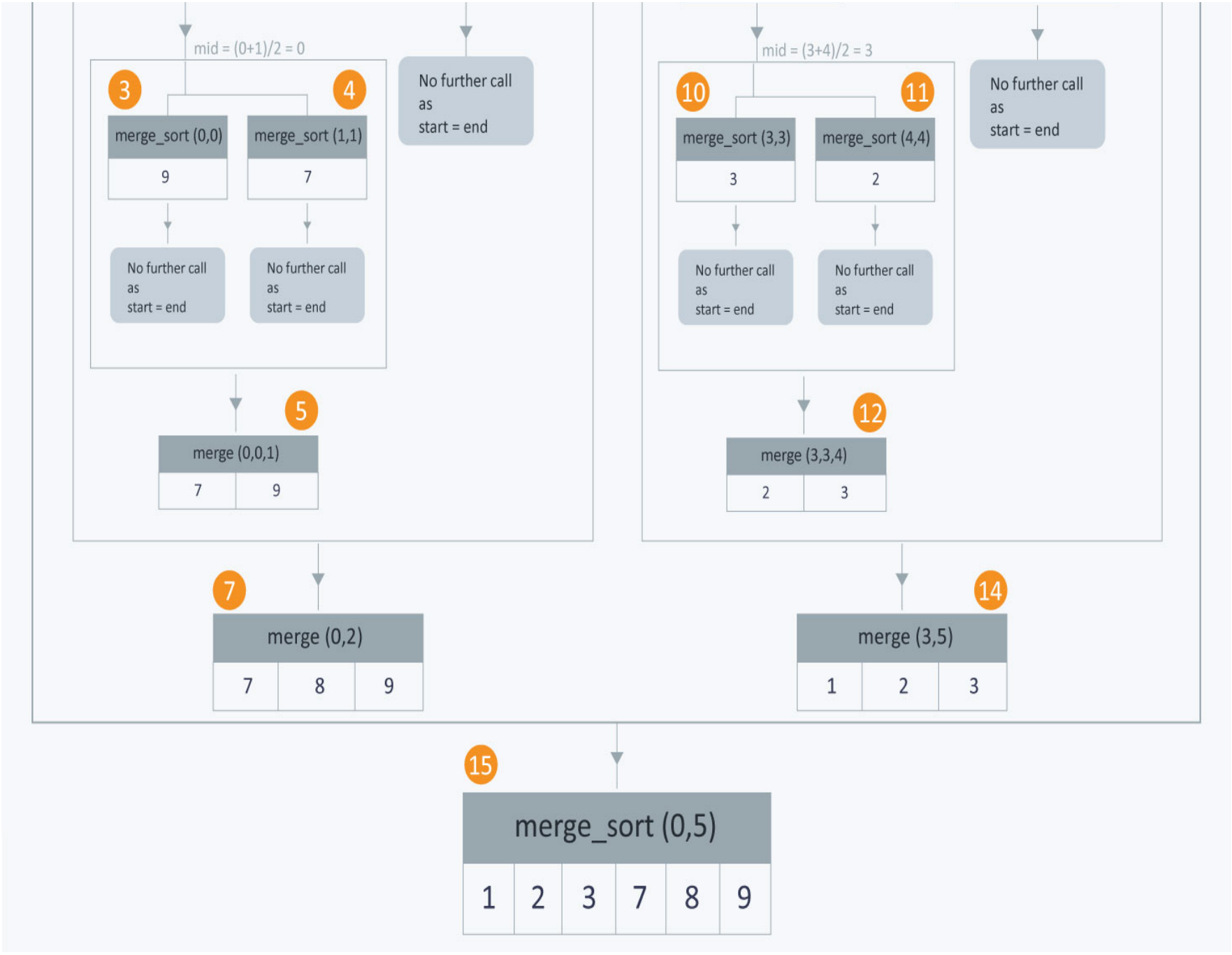
Merge Sort



Example


10 5 8 2 12 6 3 4
10 5 8 2
10 5
8 2
10
5
5 10
8
2
2 8
2 5 8 10
12 6 3 4
12 6
3 4
12
6
6 12
3
4
3 4
3 4 6 12
2 3 4 5 6 8 10 12


merge_sort (0,5)
9 7 8 3 2 1
mid = (0+5)/2 = 2
1
merge_sort (0,2)
9 7 8
8
merge_sort (3,5)
3 2 1
mid = (0+2)/2 = 1
mid = (3+5)/2 = 4
2
merge_sort (0,1)
9 7
6
merge_sort (2,2)
8
9
merge_sort (3,4)
3 2
13
merge_sort (5,5)
1


mid = (0+1)/2 = 0
3
4
merge_sort (0,0)
9
merge_sort (1,1)
7
No further call as start = end
No further call as start = end
No further call as start = end
5
merge (0,0,1)
7
9
mid = (3+4)/2 = 3
10
11
merge_sort (3,3)
3
merge_sort (4,4)
2
No further call as start = end
No further call as start = end
No further call as start = end
12
merge (3,3,4)
2
3
7
merge (0,2)
7
8
9
14
merge (3,5)
1
2
3
15
merge_sort (0,5)
1
2
3
7
8
9

Efficiency of Merge Sort

- **O (N*logN)**

Merge Sort-C code sample

```
#include<stdio.h>
void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);

int main()
{   int a[30],n,i;
   printf("Enter no of elements:");
   scanf("%d",&n);
   printf("Enter array elements:");
      for(i=0;i<n;i++)
      scanf("%d",&a[i]);

   mergesort(a,0,n-1);
      printf("\nSorted array is :");
   for(i=0;i<n;i++)
      printf("%d ",a[i]);
         return 0;  }
```

Merge sort-C code sample

```
void mergesort(int a[],int i,int j)
{
   int mid;

   if(i<j)
   {
      mid=(i+j)/2;
      mergesort(a,i,mid);       //left recursion
      mergesort(a,mid+1,j);    //right
recursion
      merge(a,i,mid,mid+1,j);    //merging of
two sorted sub-arrays
   }
}
```

```
void merge(int a[],int i1,int j1,int i2,int j2)
{
   int temp[50];    //array used for merging
   int i,j,k;
   i=i1;    //beginning of the first list
   j=i2;    //beginning of the second list
   k=0;

   while(i<=j1 && j<=j2)    //while elements in both
lists
   {
      if(a[i]<a[j])
         temp[k++]=a[i++];
      else
         temp[k++]=a[j++];
   }
      while(i<=j1)    //copy remaining elements of the
first list
      temp[k++]=a[i++];

   while(j<=j2)    //copy remaining elements of the
second list
      temp[k++]=a[j++];
         //Transfer elements from temp[] back to a[]
   for(i=i1,j=0;i<=j2;i++,j++)
      a[i]=temp[j];  }
```

Quick Sort

Quick sort

- Quicksort (sometimes called partition-exchange sort) is an efficient sorting algorithm, serving as a systematic method for placing the elements of an array in order.

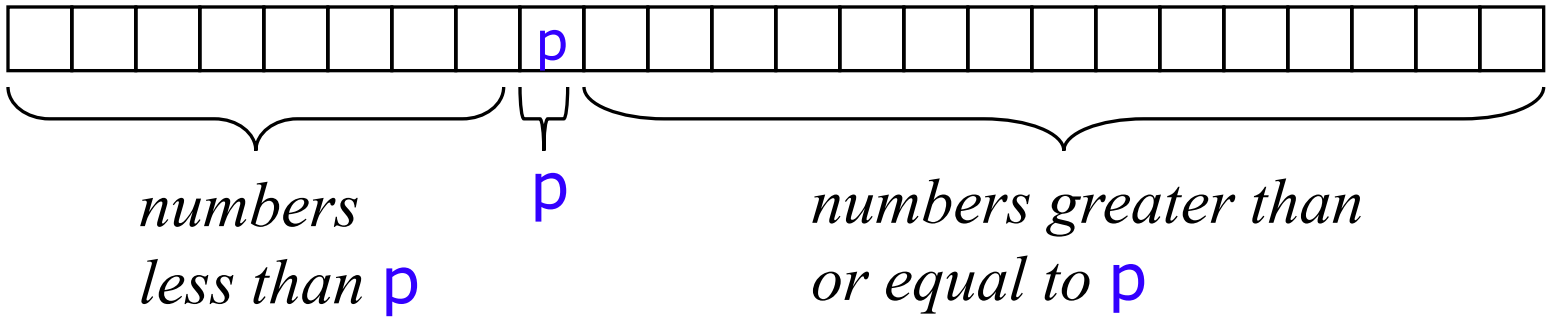
- Quicksort is a comparison sort, meaning that it can sort items of any type for which a "less-than" relation (formally, a total order) is defined.

- In efficient implementations it is not a stable sort, meaning that the relative order of equal sort items is not preserved.

- Quicksort can operate in-place on an array, requiring small additional amounts of memory to perform the sorting.

Quicksort : Basic idea

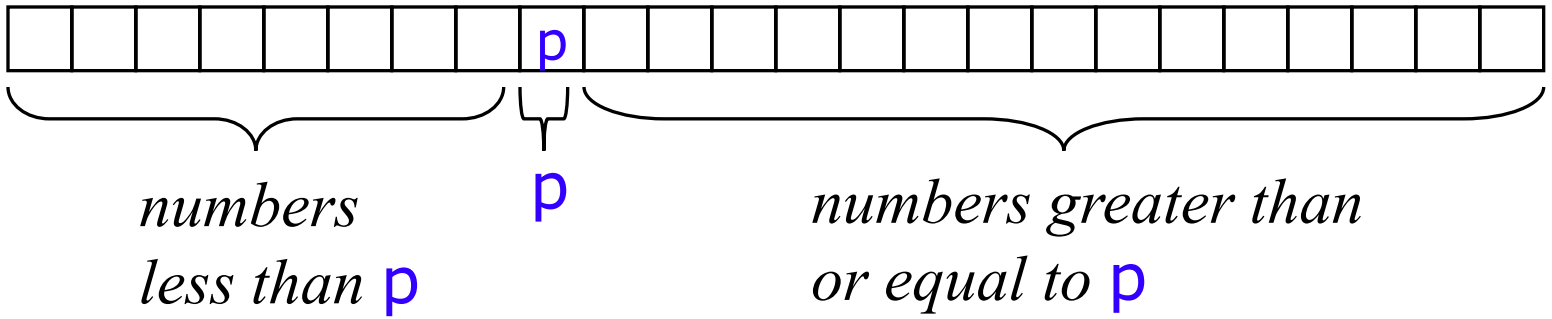
- Pick some number p from the array (this number is called Pivot)
- Move all numbers less than p to the beginning of the array
- Move all numbers greater than (or equal to) p to the end of the array
- Quicksort the numbers less than p
- Quicksort the numbers greater than or equal to p



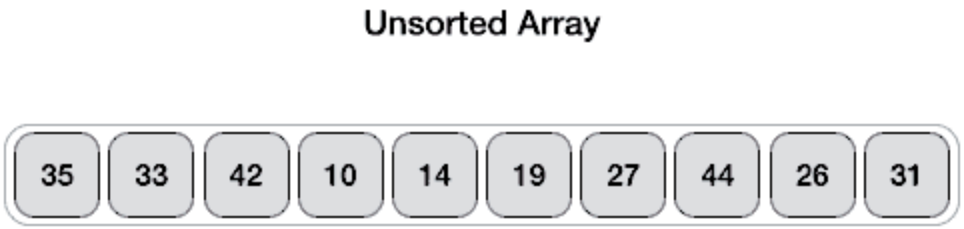
Partitioning (Quicksort II)

A key step in the Quicksort algorithm is partitioning the array

- We choose some (any) number p in the array to use as a pivot
- We partition the array into three parts:



How to select pivot



Partitioning II

- Choose an array value (say, the first) to use as the pivot
- Starting from the left end, find the first element that is greater than or equal to the pivot
- Searching backward from the right end, find the first element that is less than the pivot
- Interchange (swap) these two elements
- Repeat, searching from where we left off, until done

Partitioning

To partition a[left...right]:

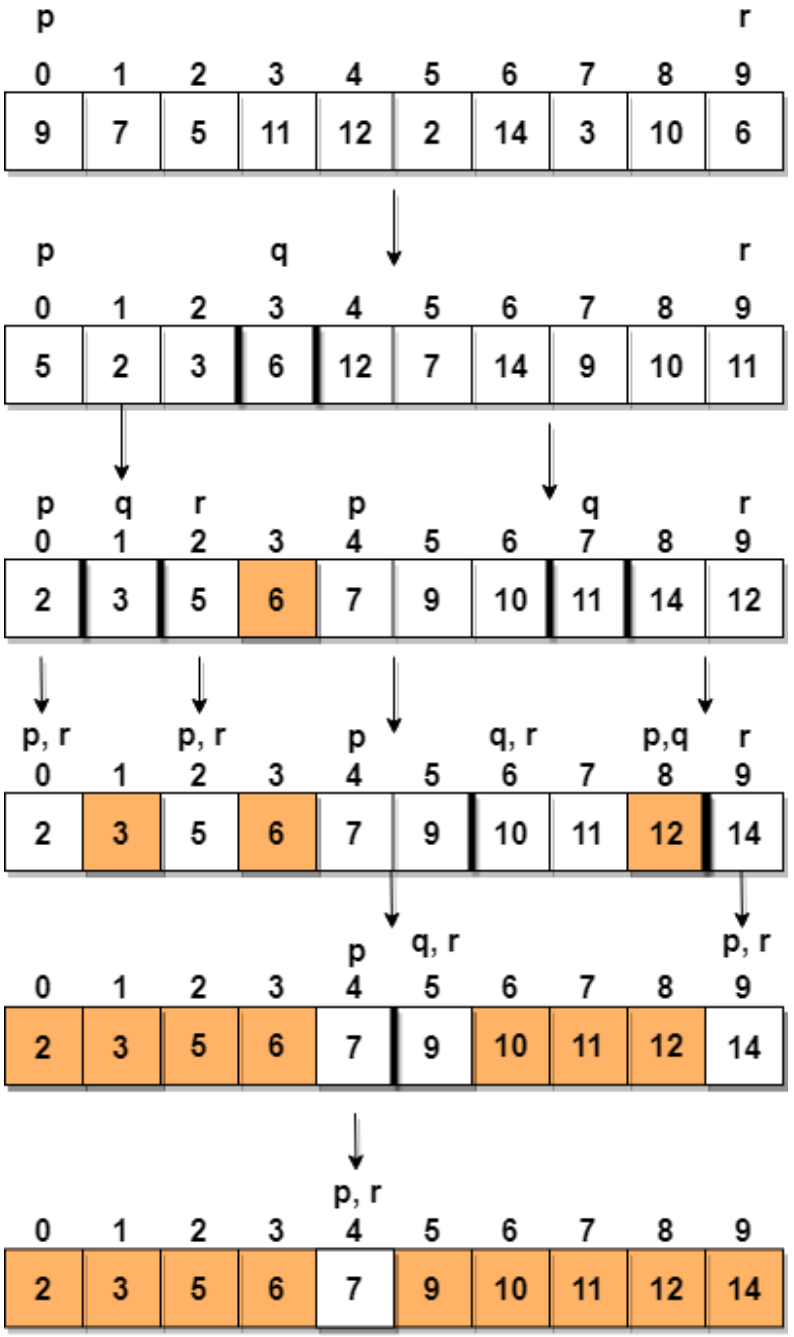
1. Set pivot = a[left], l = left + 1, r = right;
2. while l < r, do
   2.1. while l < right & a[l] < pivot , set l = l + 1
   2.2. while r > left & a[r] >= pivot , set r = r - 1
   2.3. if l < r, swap a[l] and a[r]
3. Set a[left] = a[r], a[r] = pivot
4. Terminate

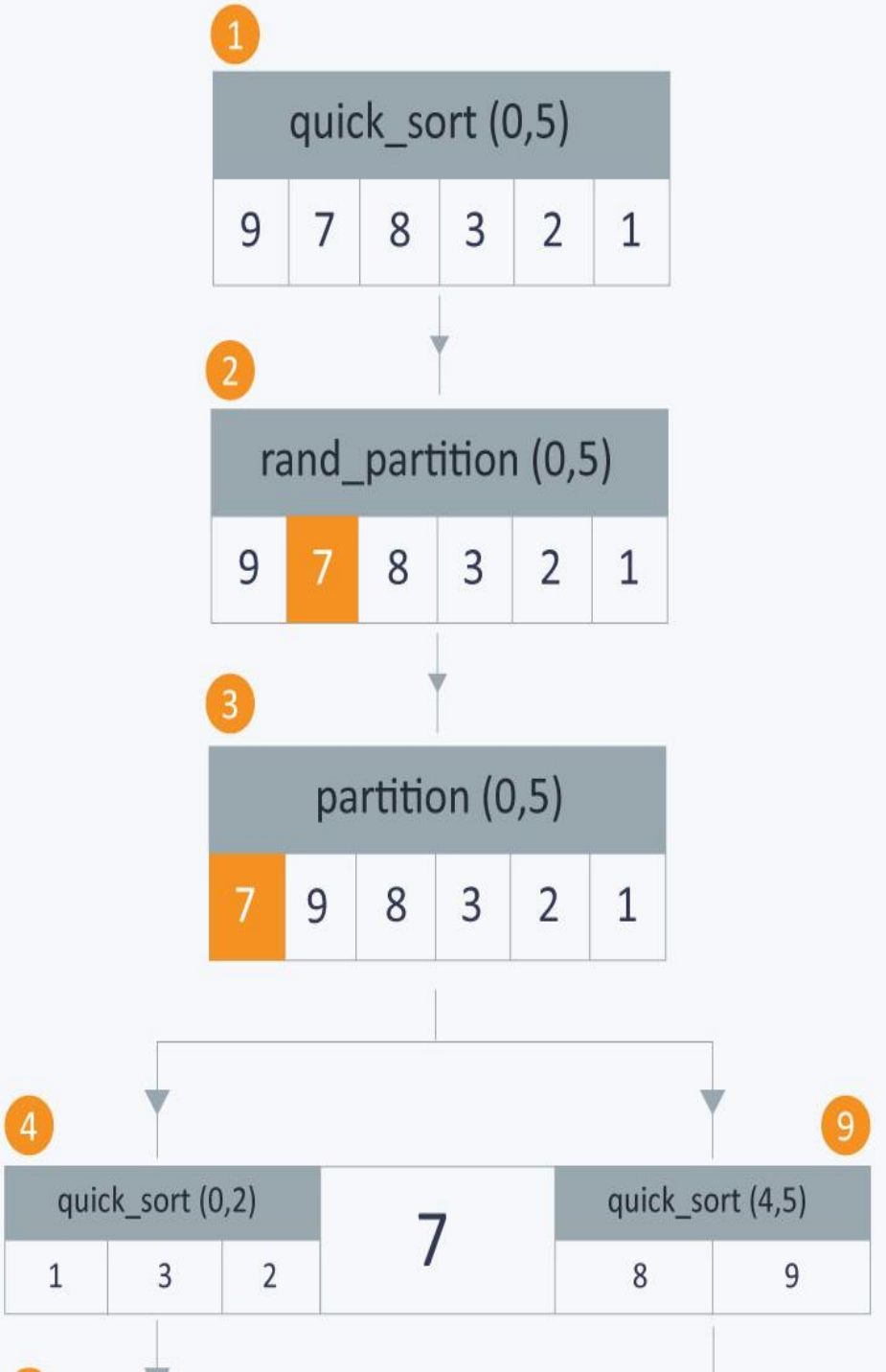
Quick Sort Algorithm

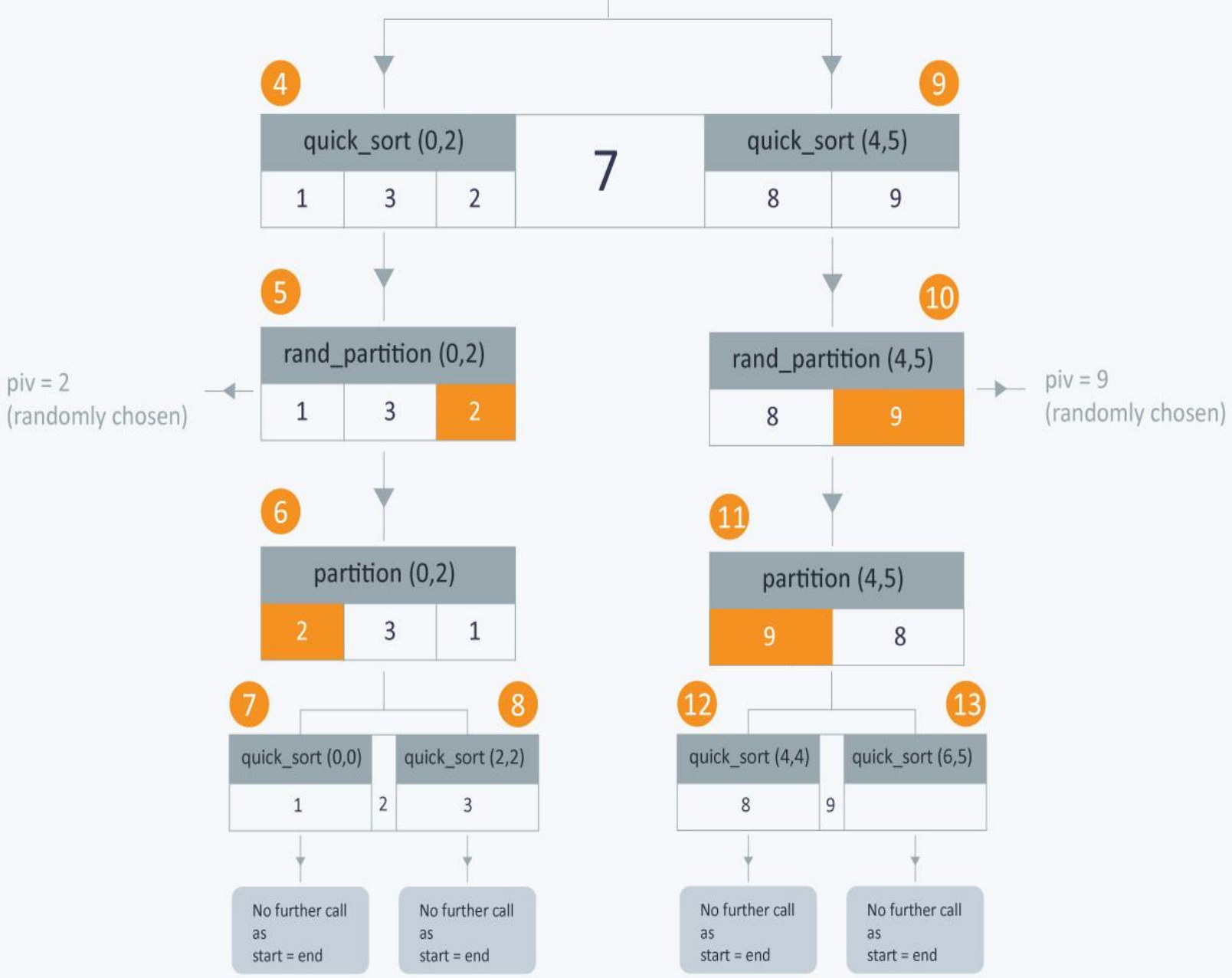
Using pivot algorithm recursively, we end up with smaller possible partitions. Each partition is then processed for quick sort. We define recursive algorithm for quicksort as follows.

- Step 1 – Make the right-most index value pivot
- Step 2 – partition the array using pivot value
- Step 3 – quicksort left partition recursively
- Step 4 – quicksort right partition recursively

Pictorial representation




1
quick_sort (0,5)
9 7 8 3 2 1
2
rand_partition (0,5)
9 7 8 3 2 1
3
partition (0,5)
7 9 8 3 2 1
4
quick_sort (0,2)
1 3 2
7
9
quick_sort (4,5)
8 9


4
quick_sort (0,2)
1
3
2
7
9
quick_sort (4,5)
8
9
5
rand_partition (0,2)
1
3
2
piv = 2
(randomly chosen)
10
rand_partition (4,5)
8
9
piv = 9
(randomly chosen)
6
partition (0,2)
2
3
1
11
partition (4,5)
9
8
7
quick_sort (0,0)
1
2
8
quick_sort (2,2)
3
12
quick_sort (4,4)
8
9
13
quick_sort (6,5)
No further call as start = end
No further call as start = end
No further call as start = end
No further call as start = end

Complexity Analysis of Quick Sort

- Worst Case Time Complexity [Big-O]: **$O(n^2)$**
- Space Complexity: **O(n*log n)**

		Time Complexity					
		Best	Worst	Avg.	Space	Stable	Comments
Comparison Sort	Bubble Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	Yes	*For each pair of indices, swap the elements if they are out of order*
	Modified Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	*At each Pass check if the Array is already sorted.Best Case-Array Already sorted*
	Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	Yes	*Swap happens only when once in a Single pass*
	Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	*Very small constant factor even if the complexity is O(n^2).* ***Best Case:*** *Array already sorted* ***Worst Case:*** *sorted in reverse order*
	Quick Sort	O(n.lg(n))	O(n^2)	O(n.lg(n))	O(1)	Yes	*Best Case: whenpivote divide in 2 equal halves Worst Case: Array already sorted - 1/n-1 partition*
	Randomized Quick Sort	O(n.lg(n))	O(n.lg(n))	O(n.lg(n))	O(1)	Yes	*Pivot choosen randomly*
	Merge Sort	O(n.lg(n))	O(n.lg(n))	O(n.lg(n))	O(n)	Yes	*Best to sort linked list (constant extra space). Best for very large number of elements which cannot fit in memory (External sorting)*
	Heap Sort	O(n.lg(n))	O(n.lg(n))	O(n.lg(n))	O(1)	No	
Non-Comparison Sort	Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(n+2^k)	Yes	*k = Range of Numbers in the list*
	Radix Sort	O(n.k/s)	O(2^s.n.k/s)	O(n.k/s)	O(n)	No	
	Bucket Sort	O(n.k)	O(n^2.k)	O(n.k)	O(n.k)	Yes	

www.ritambhara.in

Summary of Sorting Algorithms

Algorithm	Time	Notes
Selection Sort	$O(n^2)$	▪ in-place ▪ slow (good for small inputs)
Insertion Sort	$O(n^2)$	▪ in-place ▪ slow (good for small inputs)
Quicksort	$O(n*\log n)$ (expected)	▪ in-place, randomized ▪ fastest (good for large inputs)
Heapsort	$O(n*\log n)$	▪ in-place ▪ fast (good for large inputs)
Mergesort	$O(n*\log n)$	▪ sequential data access ▪ fast (good for huge inputs)

END